

ASSIGNMENT-14.4

NAME: B. MANISHWAR
HALLTICKET:2303A51276
BATCH:05

#TASK 1:

PROMPT:

Generate a responsive homepage using HTML and CSS. The page should include a header with navigation links, a main content section, and a footer. Use CSS media queries to ensure the layout adapts to mobile, tablet, and desktop screen sizes. Keep the design clean and modern.

CODE:

Index.html code:

```
index.html ×
index.html > html > body > section.features > div.feature-item
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Startup Homepage</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10   <header>
11     <h1>Startup </h1>
12     <nav>
13       <ul>
14         <li><a href="#">Home</a></li>
15         <li><a href="#">About</a></li>
16         <li><a href="#">Services</a></li>
17         <li><a href="#">Contact</a></li>
18       </ul>
19     </nav>
20   </header>
21
22   <section class="hero">
23     <h2>Welcome to Our Startup</h2>
24     <p>Innovating the future with cutting-edge technology.</p>
25     <a href="#" class="button">Get Started</a>
26   </section>
27
28   <section class="features">
29     <div class="feature-item">
30       <h3>Feature 1</h3>
31       <p></p>
32     </div>
33     <div class="feature-item">
34       <h3>Feature 2</h3>
35       <p></p>
36     </div>
37     <div class="feature-item">
38       <h3>Feature 3</h3>
39       <p></p>
40     </div>
41   </section>
42
43   <footer>
44     <p>&copy; 2023 Startup Name. All rights reserved.</p>
45     <div class="social-links">
46       <a href="#">Facebook</a>
47       <a href="#">Twitter</a>
48       <a href="#">LinkedIn</a>
49     </div>
50   </footer>
51 </body>
52 </html>
```

Style.css code:

```
# style.css > ...
1  /* Basic Reset & Body Styles */
2  body {
3      margin: 0;
4      font-family: Arial, sans-serif;
5      line-height: 1.6;
6      background-color: #f4f4f4;
7      color: #333;
8  }
9
10 .container {
11     width: 90%;
12     max-width: 1200px;
13     margin: auto;
14     overflow: hidden;
15     padding: 0 20px;
16 }
17
18 /* Header Styles */
19 header {
20     background: #333;
21     color: #fff;
22     padding: 1rem 0;
23     border-bottom: #77aaff 3px solid;
24     display: flex;
25     justify-content: space-between;
26     align-items: center;
27 }
28
29 header h1 {
30     margin: 0;
31     padding: 0;
32     flex-grow: 1;
33 }
34
35 header nav ul {
36     padding: 0;
37     list-style: none;
38     display: flex;
39 }
40
41 header nav ul li {
42     margin-left: 20px;
43 }
44
45 header nav a {
46     color: #fff;
47     text-decoration: none;
48     text-transform: uppercase;
49     font-size: 16px;
50 }
51
52 header nav a:hover {
53     color: #77aaff;
54 }
55
56 /* Hero Section */
57 .hero {
58     background: #555 url('https://via.placeholder.com/1500x500') no-repeat center center/cover;
59     color: #fff;
60     text-align: center;
61     padding: 100px 20px;
62     margin-bottom: 30px;
63     border-radius: 8px;
64 }
65
```

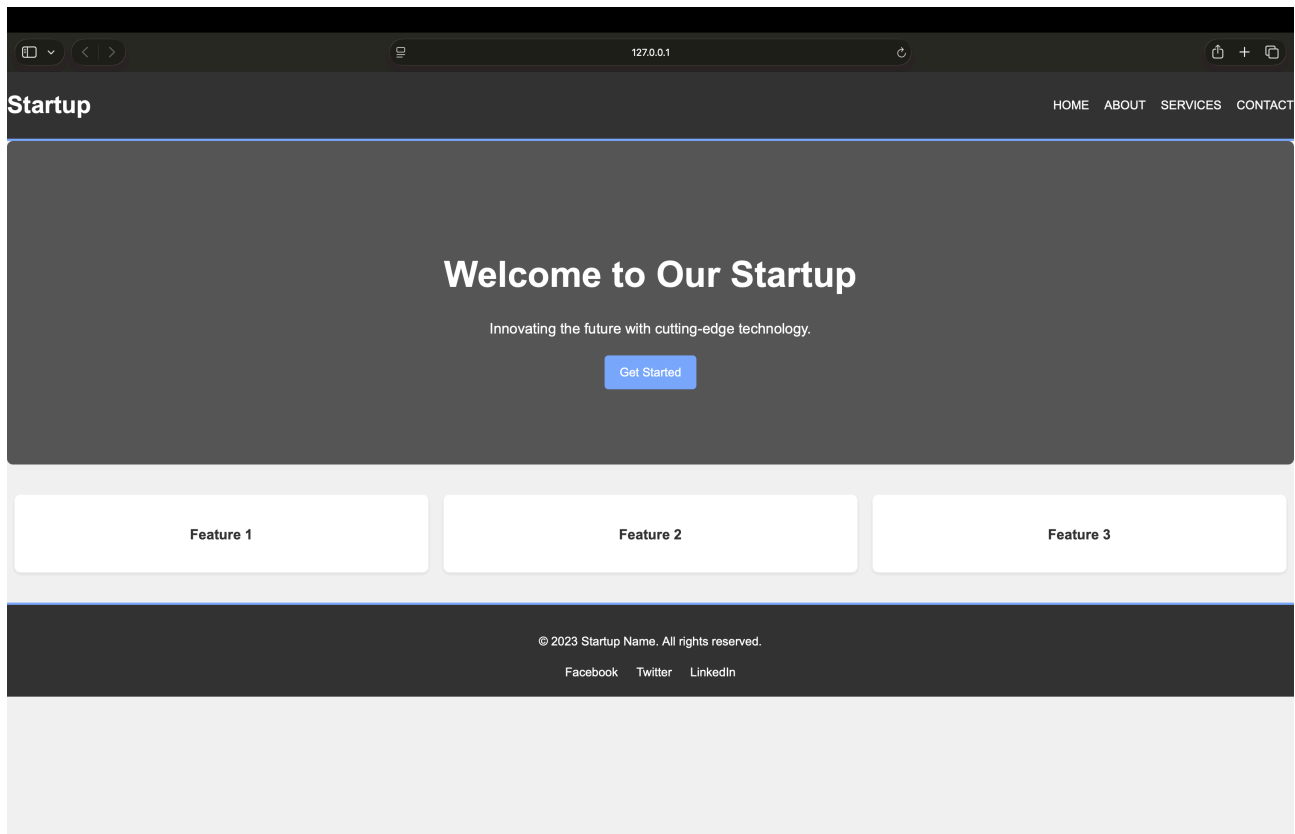
```
# style.css > ...
65
66 .hero h2 {
67     font-size: 3em;
68     margin-bottom: 10px;
69 }
70
71 .hero p {
72     font-size: 1.2em;
73     margin-bottom: 20px;
74 }
75
76 .button {
77     display: inline-block;
78     color: #fff;
79     background: #77aaff;
80     padding: 10px 20px;
81     text-decoration: none;
82     border-radius: 5px;
83     transition: background 0.3s ease;
84 }
85
86 .button:hover {
87     background: #5588dd;
88 }
89
90 /* Features Section */
91 .features {
92     display: flex;
93     justify-content: space-around;
94     flex-wrap: wrap;
95     margin-bottom: 30px;
96 }
97
98 .feature-item {
99     background: #fff;
100     padding: 20px;
101     margin: 10px;
102     border-radius: 8px;
103     box-shadow: 0 2px 5px rgba(0,0,0,0.1);
104     flex: 1;
105     min-width: 280px;
106     text-align: center;
107 }
108
109 .feature-item h3 {
110     color: #333;
111     margin-bottom: 10px;
112 }
113
114 /* Footer Styles */
115 footer {
116     background: #333;
117     color: #fff;
118     text-align: center;
119     padding: 20px 0;
120     margin-top: 30px;
121     border-top: #77aaff 3px solid;
122 }
123
124 footer .social-links a {
125     color: #fff;
126     margin: 0 10px;
127     text-decoration: none;
128 }
129
```

```

# style.css > ...
130 footer .social-links a:hover {
131   color: #77aaff;
132 }
133 /* Media Queries for Responsiveness */
134 /* Tablets (smaller than 768px) */
135 @media (max-width: 768px) {
136   header {
137     flex-direction: column;
138     text-align: center;
139   }
140
141   header nav ul {
142     margin-top: 10px;
143     justify-content: center;
144   }
145
146   .hero h2 {
147     font-size: 2.5em;
148   }
149
150   .features {
151     flex-direction: column;
152     align-items: center;
153   }
154
155   .feature-item {
156     width: 90%;
157   }
158 }
159 /* Mobile devices (smaller than 480px) */
160 @media (max-width: 480px) {
161   header h1 {
162     font-size: 1.8em;
163   }
164
165   header nav ul li {
166     margin: 0 8px;
167   }
168
169   .hero {
170     padding: 60px 15px;
171   }
172
173   .hero h2 {
174     font-size: 2em;
175   }
176
177   .hero p {
178     font-size: 1em;
179   }
180
181   .feature-item {
182     padding: 15px;
183   }
184
185   footer .social-links {
186     display: flex;
187     justify-content: center;
188     flex-wrap: wrap;
189   }
190
191   footer .social-links a {
192     margin: 5px;
193   }
194 }

```

OUTPUT:



CODE EXPLANATION:

HTML Structure

The webpage is divided into three main sections:

Header → Contains title and navigation links

Main → Displays content of the webpage

Footer → Shows copyright information

CSS Styling

Flexboxyling is applied for layout, colors, and spacing

Flexbox is used to align navigation links

TASK 2:

PROMPT:

Generate HTML with a call-to-action button.

Add JavaScript that triggers an alert message when the button is clicked.

Ensure the JavaScript code is well-commented and does not reload the page.

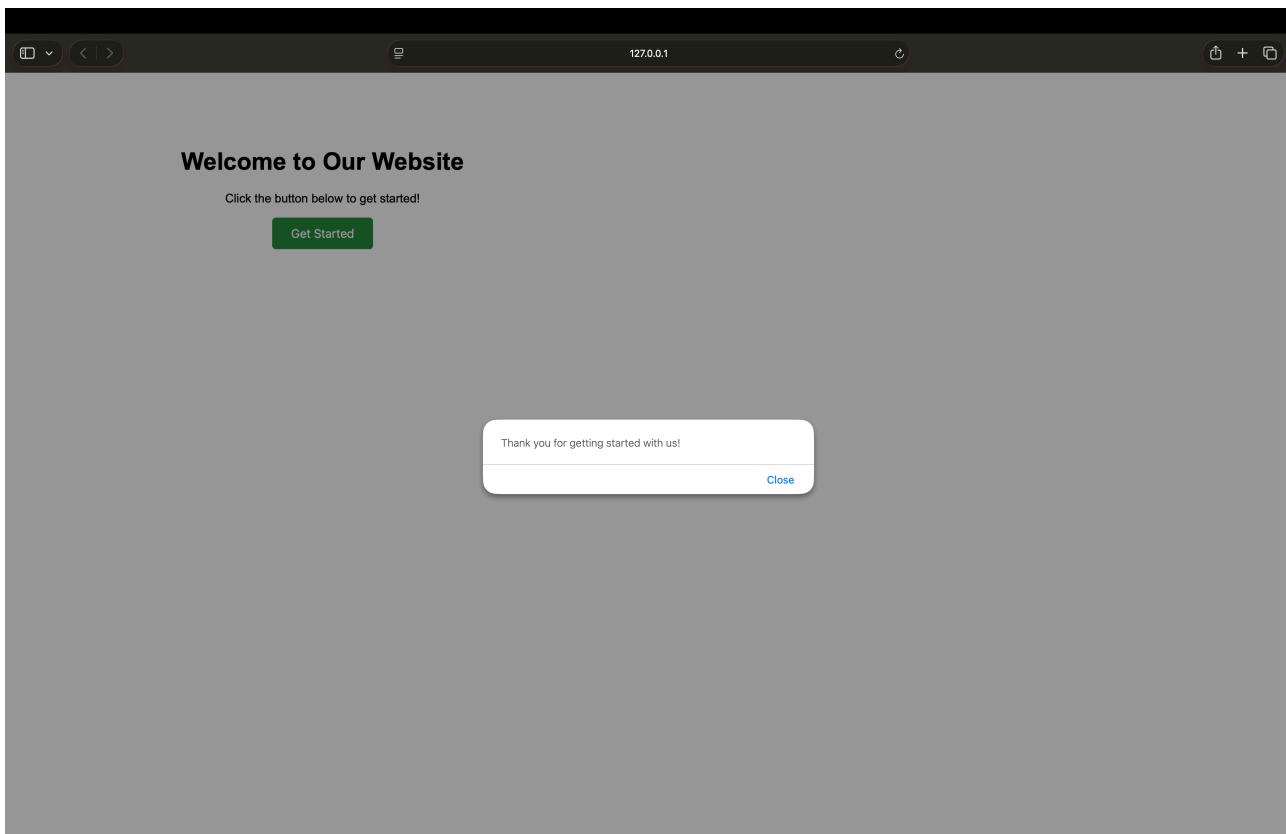
CODE:

```

<> TAST2.html > html > head > style > button:hover
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>CTA Button</title>
7
8      <style>
9          body {
10             font-family: Arial, sans-serif;
11             text-align: center;
12             margin-top: 100px;
13         }
14
15         button {
16             padding: 12px 25px;
17             font-size: 16px;
18             background-color: #28a745;
19             color: white;
20             border: none;
21             border-radius: 5px;
22             cursor: pointer;
23         }
24
25         button:hover {
26             background-color: #218838;
27         }
28     </style>
29 </head>
30 <body>
31
32     <h1>Welcome to Our Website</h1>
33     <p>Click the button below to get started!</p>
34
35     <!-- Call-To-Action Button -->
36     <button id="ctaButton">Get Started</button>
37
38     <script>
39         // Select the button using its ID
40         const button = document.getElementById("ctaButton");
41
42         // Add an event listener to detect button click
43         button.addEventListener("click", function () {
44
45             // This function runs when the button is clicked
46
47             // Display an alert message to the user
48             alert("Thank you for getting started with us!");
49
50             // Note: No page reload happens because we are not submitting a form
51         });
52     </script>
53
54 </body>
55 </html>

```

OUTPUT:



CODE EXPLANATION:

HTML CODE:

A button element is created using `<button>`

It is given an id (ctaButton) to access it in JavaScript

JavaScript Logic:

`document.getElementById()` is used to select the button

`addEventListener("click", function)` is used to detect when the button is clicked

When clicked, an alert message is displayed

#TASK 3:

PROMPT:

Create an HTML contact form with fields for name, email, and message.

Add JavaScript validation to prevent empty inputs and check for valid email format. Display inline error messages and show a success message when the form is valid. Ensure validation happens on the client side without page reload.

CODE:

```

2   <html lang="en">
46  <body>
48  <div class="form-container">
49    <h2>Contact Us</h2>
50
51    <form id="contactForm">
52
53      <!-- Name -->
54      <input type="text" id="name" placeholder="Enter your name">
55      <div id="nameError" class="error"></div>
56
57      <!-- Email -->
58      <input type="text" id="email" placeholder="Enter your email">
59      <div id="emailError" class="error"></div>
60
61      <!-- Message -->
62      <textarea id="message" placeholder="Enter your message"></textarea>
63      <div id="messageError" class="error"></div>
64
65      <button type="submit">Submit</button>
66    </form>
67
68    <!-- Success Message -->
69    <div id="successMsg" class="success"></div>
70  </div>
71  <script>
72    // Get form element
73    const form = document.getElementById("contactForm");
74
75    // Add submit event
76    form.addEventListener("submit", function(e) {
77
78      // Prevent page reload
79      e.preventDefault();
80
81      // Get input values
82      const name = document.getElementById("name").value.trim();
83      const email = document.getElementById("email").value.trim();
84      const message = document.getElementById("message").value.trim();
85
86      // Get error elements
87      const nameError = document.getElementById("nameError");
88      const emailError = document.getElementById("emailError");
89      const messageError = document.getElementById("messageError");
90      const successMsg = document.getElementById("successMsg");
91
92      // Clear previous errors
93      nameError.textContent = "";
94      emailError.textContent = "";
95      messageError.textContent = "";
96      successMsg.textContent = "";
97
98      let isValid = true;
99
100     // Name validation
101     if (name === "") {
102       nameError.textContent = "Name is required";
103       isValid = false;
104     }
105
106     // Email validation
107     const emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
108
109     if (email === "") {
110       emailError.textContent = "Email is required";

```

```

    </body>
    <script>
        form.addEventListener("submit", function(e) {

            if (email === "") {
                emailError.textContent = "Email is required";
                isValid = false;
            } else if (!email.match(emailPattern)) {
                emailError.textContent = "Enter a valid email";
                isValid = false;
            }

            // Message validation
            if (message === "") {
                messageError.textContent = "Message cannot be empty";
                isValid = false;
            }

            // If all valid
            if (isValid) {
                successMsg.textContent = "Form submitted successfully!";
                form.reset(); // Clear form
            }

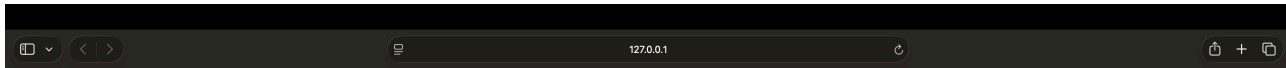
        });
    </script>

```

OUTPUT:
For Valid Submission:

The screenshot shows a web browser window with the address bar displaying '127.0.0.1'. The page content includes a heading 'Contact Us' followed by three text input fields with placeholder text: 'Enter your name', 'Enter your email', and 'Enter your message'. Below the inputs is a blue 'Submit' button. At the bottom, a green message reads 'Form submitted successfully!'.

For Invalid Submission:



Contact Us

Enter a valid email

CODE EXPLANATION:

Form Design

The form includes three input fields:

Name

Email

Message

Validation Logic

Name → must not be empty

Email → must follow correct format using regex

Message → must not be empty

Error Handling

Error messages are displayed below each field

Previous errors are cleared before new validation

#TASK 4:

PROMPT:

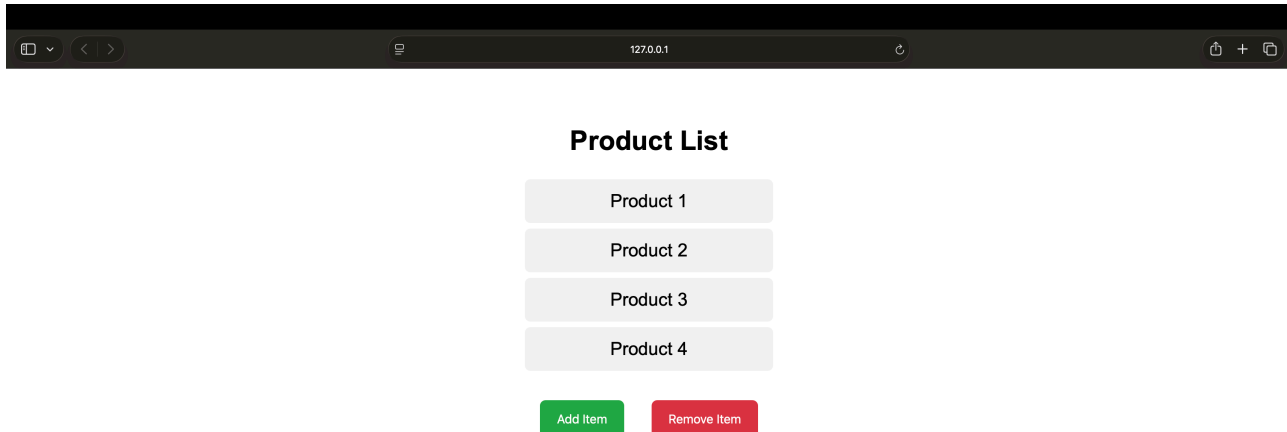
Create an HTML page with a list of items. Add JavaScript functionality to dynamically add a new item when an "Add" button is clicked and remove the last item when a "Remove" button is clicked. Ensure the DOM updates instantly without page reload and include comments for clarity.

CODE:

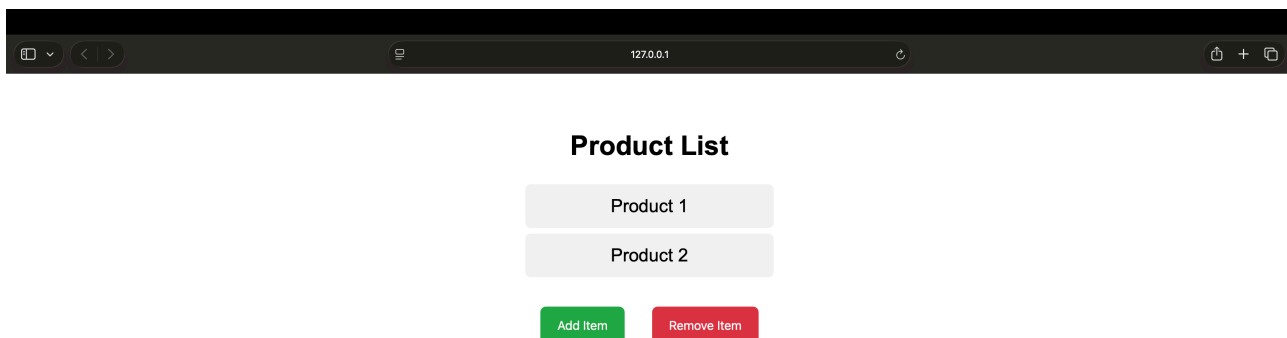
```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Dynamic List</title>
7
8   <style>
9     body {
10       font-family: Arial, sans-serif;
11       text-align: center;
12       margin-top: 50px;
13     }
14
15     ul {
16       list-style: none;
17       padding: 0;
18     }
19
20     li {
21       background: #f4f4f4;
22       margin: 5px auto;
23       padding: 10px;
24       width: 200px;
25       border-radius: 5px;
26     }
27
28     button {
29       margin: 10px;
30       padding: 10px 15px;
31       border: none;
32       cursor: pointer;
33       border-radius: 5px;
34     }
35
36     .add-btn {
37       background: #28a745;
38       color: white;
39     }
40
41     .remove-btn {
42       background: #dc3545;
43       color: white;
44     }
45   </style>
46 </head>
47 <body>
48   <h2>Product List</h2>
49
50   <!-- List -->
51   <ul id="itemList">
52     <li>Product 1</li>
53     <li>Product 2</li>
54   </ul>
55
56   <!-- Buttons -->
57   <button class="add-btn" onclick="addItem()">Add Item</button>
58   <button class="remove-btn" onclick="removeItem()">Remove Item</button>
59
60   <script>
61     // Counter to keep track of item numbers
62     let count = 3;
63
64     // Function to add a new item
65     function addItem() {
66       // Select the list
67       const list = document.getElementById("itemList");
68
69       // Create a new list item
70       const newItem = document.createElement("li");
71
72       // Set text for new item
73       newItem.textContent = "Product " + count;
74
75       // Append item to list
76       list.appendChild(newItem);
77
78       // Increment counter
79       count++;
80     }
81
82     // Function to remove last item
83     function removeItem() {
84       const list = document.getElementById("itemList");
85
86       // Check if list has items
87       if (list.children.length > 0) {
88         // Remove last child
89         list.removeChild(list.lastElementChild);
90         count--;
91       } else {
92         alert("No items to remove!");
93       }
94     }
95   </script>
96 </body>
97 </html>
```

OUTPUT:

Adds a new item when an “Add” button is clicked:



Removes the last item when a “Remove” button is clicked:



CODE EXPLANATION:

HTML Structure

An unordered list is used to display items

Two buttons:

Add Item

Remove Item

DOM Manipulation

The list updates instantly without page reload

JavaScript directly modifies the DOM (Document Object Model)

- **Created a dynamic product list**
- **Used JavaScript DOM manipulation**
- **Enabled real-time updates without refresh**
- **Implemented add/remove functionality**

#TASK 5:

PROMPT:

Generate a modal popup using HTML, CSS, and JavaScript. Include a semi-transparent background overlay. Add functionality to open the modal using a button and close it using a close button or by clicking outside the modal.

Ensure the code is clean and well-commented.

CODE:

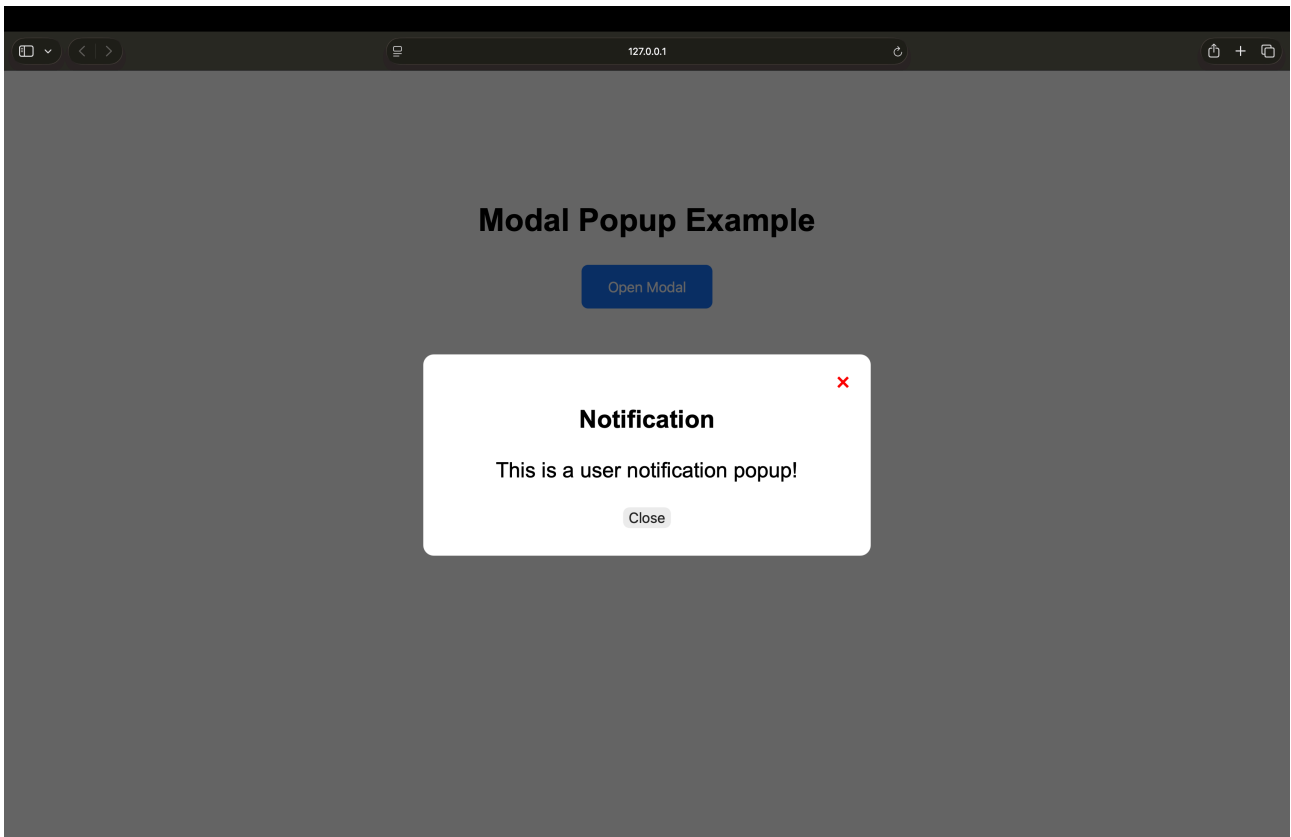
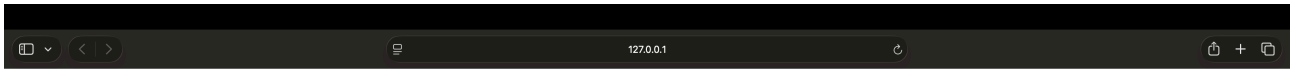
```

2   <html lang="en">
58  <body>
59
60   <h2>Modal Popup Example</h2>
61
62   <!-- Button to open modal -->
63   <button class="open-btn" onclick="openModal()">Open Modal</button>
64
65   <!-- Modal Structure -->
66   <div id="modal" class="modal">
67
68       <!-- Modal Content -->
69       <div class="modal-content">
70
71           <!-- Close Button -->
72           <span class="close" onclick="closeModal()">&times;</span>
73
74           <h3>Notification</h3>
75           <p>This is a user notification popup!</p>
76
77           <button onclick="closeModal()">Close</button>
78       </div>
79   </div>
80
81   <script>
82       // Select modal element
83       const modal = document.getElementById("modal");
84
85       // Function to open modal
86       function openModal() {
87           modal.style.display = "flex"; // Show modal
88       }
89
90       // Function to close modal
91       function closeModal() {
92           modal.style.display = "none"; // Hide modal
93       }
94
95       // Close modal when clicking outside content (overlay click)
96       window.onclick = function(event) {
97           if (event.target === modal) {
98               closeModal();
99           }
100       };
101   </script>
102
103 </body>
104 </html>

```

OUTPUT:

UserNotification Popup



CODE EXPLANATION:
HTML Structure

A button is used to open the modal

A <div> with class modal acts as overlay

Inside it, modal-content contains popup message

A close button (×) is added

JavaScript Logic

➤ Open Modal

Changes display from none → flex

➤ Close Modal

Changes display from flex → none

➤ Overlay Click

If user clicks outside modal content → modal closes

Features Implemented

- Open modal using button
- Close modal using:
 - Close icon (×)
 - Close button
 - Clicking outside (overlay)